

Input and Output

Where We Left Off

We can store, compute, repeat, and organize

*But every result has stayed **inside** the program*

*So far the only way to see it has been
the **debugger***

Today: talking to the outside world

What I/O Is

Input: *data coming **into** the program*
keyboard, files, sensors, network

Output: *data going **out** of the program*
screen, files, actuators, network

*I/O is how a program serves its **users***

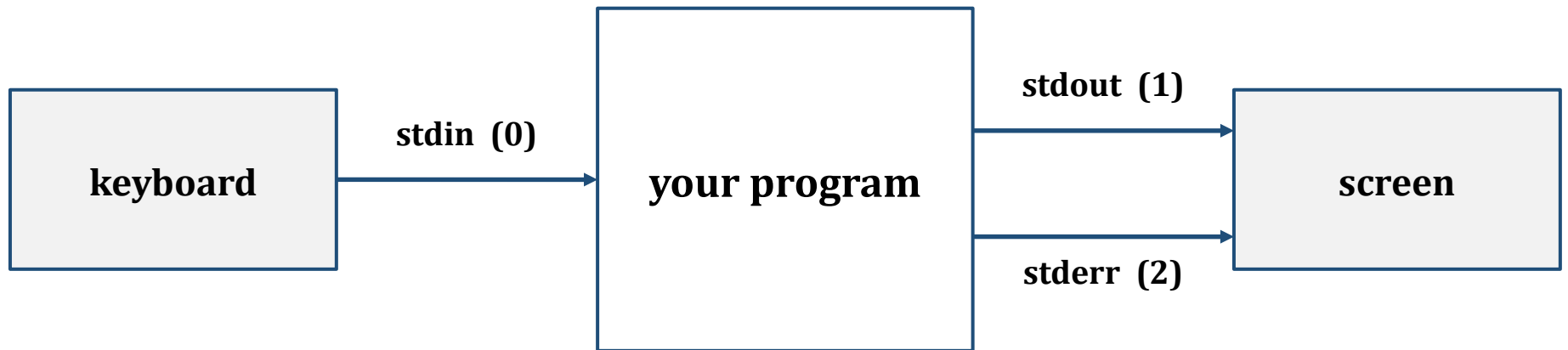
Streams

*A **stream** is a sequence of bytes flowing into or out of a program*

*Every program starts with **three**:*

<i>stdin</i>	<i>0</i>	<i>standard input</i>
<i>stdout</i>	<i>1</i>	<i>standard output</i>
<i>stderr</i>	<i>2</i>	<i>standard error</i>

The Three Standard Streams



By default all three are connected to the terminal

Why a Separate Error Stream?

./prog > results.txt
stdout goes to the file
*stderr **still** appears on the screen*

./prog > results.txt 2> errors.txt
each stream to its own file

Data and error messages never mix

Output in C: printf

```
#include <stdio.h>
```

```
printf( "Oil pressure\n" );  
printf( "psi = %f\n", psi );  
printf( "%u counts = %.1f psi\n", counts, psi );
```

% marks where a value is inserted

\n ends the line

Format Specifiers

<i>%d</i>	<i>signed int</i>	<i>%x</i>	<i>hexadecimal</i>
<i>%u</i>	<i>unsigned int</i>	<i>%c</i>	<i>one character</i>
<i>%ld</i>	<i>long</i>	<i>%s</i>	<i>string</i>
<i>%f</i>	<i>float or double</i>	<i>%p</i>	<i>pointer address</i>
<i>%e</i>	<i>scientific</i>	<i>%%</i>	<i>a literal %</i>

*The specifier must **match** the type*

Width and Precision

<code>printf("[%5d]", 42);</code>	<code>[42]</code>
<code>printf("[% -5d]", 42);</code>	<code>[42]</code>
<code>printf("[%05d]", 42);</code>	<code>[00042]</code>
<code>printf("[% .1f]", 50.0);</code>	<code>[50.0]</code>
<code>printf("[%8.2f]", 3.14159);</code>	<code>[3.14]</code>
<code>printf("[%#x]", 255);</code>	<code>[0xff]</code>

Width pads, precision sets digits after the point

Printing Fixed-Width Types

uint32_t may not be unsigned int on every machine

#include <inttypes.h>

printf("%" PRIu32 "\n", counts);

printf("%" PRIu64 "\n", timestamp_ms);

PRIu32 expands to the right specifier

A Mismatch Is a Bug

<i>printf("%d\n", 3.5);</i>	<i>prints garbage</i>
<i>printf("%s\n", 42);</i>	<i>may crash</i>

printf trusts the format string completely

*Compile with **-Wall** and the compiler
checks every format for you*

More C Output Functions

fprintf(stderr, "sensor fault\n");

to stderr

puts("done");

string + newline

putchar('x');

one character

sprintf(buf, "%.1f", psi);

into a string

snprintf(buf, sizeof buf, "%.1f", psi);

*Prefer **snprintf** - it cannot overflow buf*

Output in C++: iostream

```
#include <iostream>
```

```
std::cout << "Oil pressure\n";
```

```
std::cout << "psi = " << psi << '\n';
```

```
std::cerr << "sensor fault\n";
```

<< sends each value to the stream in turn

Formatting in C++

```
#include <iomanip>
```

```
std::cout << std::fixed << std::setprecision ( 1 )  
<< psi << '\n'; 50.0
```

```
std::cout << std::setw ( 8 ) << counts ; 1456  
std::cout << std::hex << 255 ; ff
```

*Careful: most settings **stay on** until changed*

Modern C++: `std::format`

C++20:

```
#include <format>  
std::cout << std::format ( "{:.1f} psi\n", psi ) ;
```

C++23:

```
#include <print>  
std::println ( "{:.1f} psi", psi ) ;
```

*Checked at **compile time**, like Rust*

Output in Rust: println!

```
println! ( "Oil pressure" );  
println! ( "psi = {}", psi );  
println! ( "{} counts = {:.1} psi", counts, psi );
```

<i>print!</i>	<i>no newline</i>
<i>eprintln!</i>	<i>to stderr</i>

*The ! means a **macro**, checked at compile time*

Rust Format Strings

<code>{:.1}</code>	<code>50.0</code>	<i>one decimal place</i>
<code>{:8.2}</code>	<code>3.14</code>	<i>width 8</i>
<code>{:<8}</code>	<code>42</code>	<i>left align</i>
<code>{:08}</code>	<code>00000042</code>	<i>pad with zeros</i>
<code>{:x}</code> <code>{:#x}</code>	<code>ff</code> <code>0xff</code>	<i>hexadecimal</i>
<code>{:b}</code>	<code>11111111</code>	<i>binary</i>

Name the variable inline: `println! ( "{psi:.1}" );`

Printing a Struct in Rust

```
#[derive(Debug)]  
struct Sample { adc_counts : u32, psi : f32 }  
  
println! ( "{:?}", my_sample );  
    Sample { adc_counts: 1456, psi: 50.0 }  
  
{:#?} prints one field per line
```

Printing a Struct in C and C++

C: write a function

```
void print_sample ( const sample_t *s ) {  
    printf ( "%u counts, %.1f psi\n",  
            s->adc_counts, s->psi );  
}
```

C++: overload operator<< so that

std::cout << my_sample ; works

One Line, Three Languages

C: *printf ("%.1f psi\n", psi) ;*

C++: *std::cout << std::fixed << std::setprecision (1)*
 << psi << " psi\n" ;

Rust: *println! ("{:.1} psi", psi) ;*

Output Is Buffered

*Output is collected in a **buffer** and sent in chunks*

stdout to a terminal:

sent at each newline

stdout to a file:

sent when the buffer fills

stderr:

sent immediately

Force it out:

`fflush ( stdout ) ;`

`std::cout << std::flush ;`

Buffering Can Lie to You

```
printf( "got here" );           no newline  
crash ( );
```

The program crashes - and "got here"
*is **never printed***

It was still waiting in the buffer
*You conclude the crash happened **earlier***

Print Is Not a Debugger

*Output is for the program's **users***

*It is **not** a substitute for the debugger*

Why not?

The Temptation

```
printf( "here 1\n" );  
x = compute ( a, b );  
printf( "here 2, x = %d\n", x );  
y = scale ( x );  
printf( "here 3, y = %d\n", y );
```

It feels quick. It is not.

What Goes Wrong

*Every new question means **edit, compile, run***

*You only see what you **guessed** to print*

Buffered output can vanish in a crash

Forgotten prints end up in the final program

*Printing **changes** the program you are testing*

Heisenbugs

*Add a print statement - and the bug **disappears***

The print changed:

the timing of the program

the layout of the stack

what the optimizer did

Remove the print - and the bug is back

On a Microcontroller

*Printing goes out a **serial port***

115200 baud = about 11,500 characters per second

*One 40-character line = about **3.5 ms***

Enough to break real-time timing

Never print inside an interrupt routine

Ask the Debugger Instead

The question	The debugger answer
<i>Did the program get here?</i>	<i>break file:line</i>
<i>What is x right now?</i>	<i>print x</i>
<i>What are all the locals?</i>	<i>info locals</i>
<i>How did I get here?</i>	<i>backtrace</i>
<i>Who changed x?</i>	<i>watch x</i>
<i>Only on pass 97</i>	<i>break ... if i == 97</i>
<i>Show sum at every stop</i>	<i>display sum</i>

The Same Question, Two Ways

With print: *add printf("i=%d sum=%d\n", i, sum);*
 recompile, rerun, read 100 lines

With gdb: *break main.c:12 if i == 97*
 display sum
 run

No edit. No recompile. One stop, exactly where you asked.

Our Rule

Print *statements are for the program's* **users**

Debugging *is done with the* **debugger**

Any print added while debugging is
removed *before you submit*

When Output Is the Right Tool

Logging *in a system that runs for days
or in the field, where no debugger is attached*

*Planned, not sprinkled:
a log file or stderr
severity levels: error, warning, info
can be switched off*

A design decision - not a debugging shortcut

Input: Reading From the Keyboard

*The program **waits** until the user types a line and presses Enter*

*What arrives is **text** - characters*

*It must be **converted** to a number*

And the user may type anything

Input in C: scanf

```
int count ;  
scanf ( "%d", &count ) ;
```

scanf needs the **address** of *count*
so it can store into (*count*)

```
float:      scanf ( "%f", &psi ) ;  
double:    scanf ( "%lf", &value ) ;      not %f!
```

scanf Pitfalls

Returns how many values it converted:

```
if ( scanf ( "%d", &count ) != 1 ) { /* bad input */ }
```

*Bad input stays in the buffer - the next
scanf fails on it too*

Strings: always give a width

```
char name[20]; scanf ( "%19s", name );
```

Safer Input in C

Read a whole line, then convert it:

```
char line[64];  
if ( fgets ( line, sizeof line, stdin ) != NULL ) {  
    char *end;  
    long v = strtol ( line, &end, 10 );  
    if ( end == line ) { /* not a number */  
    }
```

Input in C++: cin

```
int count ;  
if ( std::cin >> count ) {  
    // got a number  
} else {  
    // not a number - the stream is now in a fail state  
}
```

Whole line: std::getline (std::cin, line) ;

Input in Rust

```
let mut line = String::new ( ) ;  
std::io::stdin ( ).read_line ( &mut line )  
    .expect ( "read failed" ) ;
```

```
let count : i32 = line.trim ( ).parse ( )  
    .expect ( "not a number" ) ;
```

&mut *line* - *read_line* may change it

Handling Bad Input in Rust

```
match line.trim ().parse::<i32> () {  
    Ok ( n )      => println! ( "got {}", n ),  
    Err ( _ )     => eprintln! ( "not a number" ),  
}
```

*parse returns a **Result**: Ok with a value,
or Err with the reason*

*The compiler makes you handle **both***

Input Can Always Fail

The user types letters instead of digits

The input ends (Ctrl-D, end of file)

The file does not exist

The sensor is unplugged

Always check:

C return value

C++ stream state

Rust Result

Reading Until the End

C:

```
while ( fgets ( line, sizeof line, stdin ) != NULL ) { ... }
```

C++:

```
while ( std::getline ( std::cin, line ) ) { ... }
```

Rust:

```
for line in std::io::stdin ( ).lines ( ) {  
    let line = line.expect ( "read failed" );  
}
```

Files Are Streams Too

Open *a file to get a stream*

Read or write *it like stdin and stdout*

Close *it when you are done*

Everything you learned about printing applies

Writing a File in C

```
FILE *fp = fopen ( "log.csv", "w" );  
if ( fp == NULL ) {  
    perror ( "log.csv" );  
    return 1 ;  
}  
fprintf ( fp, "%u,%.1f\n", counts, psi ) ;  
fclose ( fp ) ;
```

fopen Modes

"r" read - the file must exist

*"w" write - create, or **erase** an existing file*

"a" append - add to the end

"r+" read and write

*Add **b** for binary: "rb" "wb"*

Reading a File in C

```
FILE *fp = fopen ( "log.csv", "r" );  
if ( fp == NULL ) { perror ( "log.csv" ); return 1 ; }  
  
char line[128] ;  
while ( fgets ( line, sizeof line, fp ) != NULL ) {  
    unsigned counts ; float psi ;  
    if ( sscanf ( line, "%u,%f", &counts, &psi ) == 2 ) { ... }  
}  
fclose ( fp ) ;
```

Files in C++

```
#include <fstream>
```

```
std::ofstream out ( "log.csv" );
```

```
if ( !out ) { std::cerr << "cannot open\n" ; return 1 ; }
```

```
out << counts << ' ' << psi << '\n' ;
```

*Closed **automatically** at the closing brace*

Files in Rust

```
use std::fs::File ;  
use std::io::Write ;
```

```
let mut f = File::create ( "log.csv" )? ;  
writeln! ( f, "{}{:1}", counts, psi )? ;
```

? passes any error back to the caller

*Closed **automatically** when *f* goes out of scope*

Always Close Your Files

*C: forget fclose and buffered data may
 never reach the disk*

*C++, Rust: the file closes when its owner
 goes out of **scope***

*The operating system limits how many
files can be open at once*

Example: Log Samples in C

```
FILE *fp = fopen ( "log.csv", "w" );  
if ( fp == NULL ) { perror ( "log.csv" ); return 1 ; }  
  
fprintf ( fp, "adc_counts,psi\n" );  
for ( int i = 0 ; i < n ; i++ ) {  
    fprintf ( fp, "%" PRIu32 ",%.1f\n",  
             history[i].adc_counts, history[i].psi );  
}  
fclose ( fp );
```

Example: Log Samples in Rust

```
fn save ( history : &[Sample] ) -> std::io::Result<()> {  
    let mut f = File::create ( "log.csv" )? ;  
    writeln! ( f, "adc_counts,psi" )? ;  
    for s in history.iter ( ) {  
        writeln! ( f, "{},{:.1}", s.adc_counts, s.psi )? ;  
    }  
    Ok ( () )  
}
```

Text or Binary?

Text: `fprintf ( fp, "%u,%.1f\n", ... ) ;`
human readable, portable, larger

Binary: `fwrite ( &s, sizeof s, 1, fp ) ;`
the raw bytes of the struct
*includes **padding**, depends on byte order*

Prefer text unless size or speed demands binary

I/O on a Microcontroller

No screen and no keyboard

*I/O means **pins**, the **ADC**, and a **serial port***

Serial.println (psi) ; Arduino output

*Many boards, including the ESP32-S3, have a **hardware debugger** built in over USB*

The Same Rule on Hardware

*Serial output is for **monitoring** the device*

*Debugging uses the **hardware debugger**:*

breakpoints on the real chip

watch registers and variables

no change to the program's timing

Print statements are still not a debugger

Output Compared

	C	C++	Rust
to stdout	<i>printf</i>	<i>std::cout <<</i>	<i>println!</i>
to stderr	<i>fprintf (stderr, ...)</i>	<i>std::cerr <<</i>	<i>eprintln!</i>
format	<i>%d %.1f %x</i>	<i>omanip / std::format</i>	<i>{ } {:.1} {x}</i>
to a file	<i>fprintf (fp, ...)</i>	<i>std::ofstream</i>	<i>writeln! (f, ...)</i>
to a string	<i>snprintf</i>	<i>std::format</i>	<i>format!</i>
flush	<i>fflush</i>	<i>std::flush</i>	<i>.flush ()</i>

Input Compared

	C	C++	Rust
a number	<i>scanf ("%d", &n)</i>	<i>std::cin >> n</i>	<i>read_line + parse</i>
a line	<i>fgets</i>	<i>std::getline</i>	<i>read_line</i>
convert	<i>strtol, sscanf</i>	<i>std::stoi</i>	<i>.parse ()</i>
open a file	<i>fopen</i>	<i>std::ifstream</i>	<i>File::open</i>
check it	<i>return value</i>	<i>stream state</i>	<i>Result</i>
close	<i>fclose</i>	<i>end of scope</i>	<i>end of scope</i>

What We Can Now Do

Print *formatted results in all three languages*

Read *and check user input*

Read and write *files*

*And keep print statements for users -
the **debugger** is for debugging*

Let's practice

*Exercise: Using **iopractice***